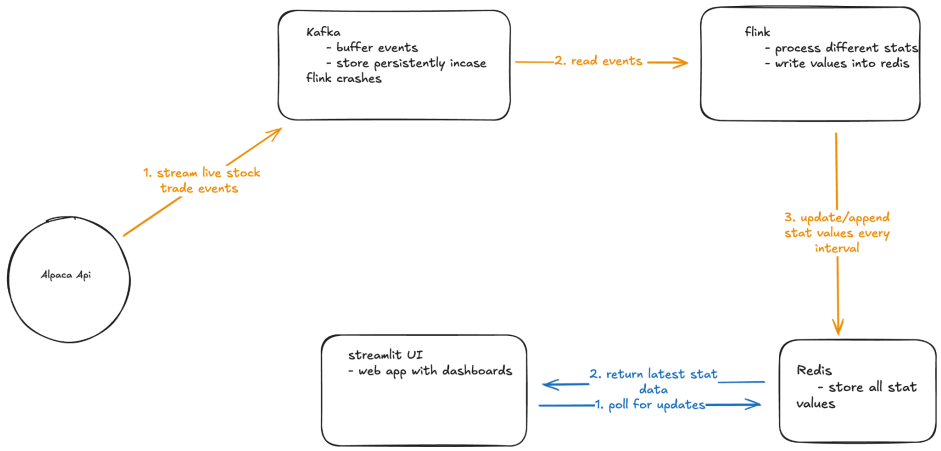
TickerFlow

authored by Samuel (Sami) Woldegiorgies

Project Description

Tickerflow is a web app-based dashboard that shows the stock price of some 30 stocks and other financial stats based on the movement of these stocks throughout the day. This is achieved on a high level using a real-time data feed of the stocks from the Alpaca API. We buffer these events using Kafka (this is feeling overkill, but I do want to learn how to use the technology, and using queues and handling retries, etc, sounded like a pain). We then feed this stream of data into Flink, where we process the stream of events to calculate some other metrics like simple weighted average, volume weighted average, candlestick data, and so on. Most of these metrics require Flink to maintain some kind of state over some window of time to get their values, which makes it a good use case to show Flink's capabilities. These stats will be written into Redis at some interval and can be used as a source for the web app to poll on to refresh the dashboards. I'm choosing Redis since it's an in-memory datastore, making it really quick to access and write to, which we need as Flink updates are frequent and the polls from the webapp are also frequent (think 1- 3s). All these different stats will be displayed on a web app, either using a Python library such as Streamlit or a JS library, if time allows.

Note: I have the free tier for the live feed, so stock data is restricted to 30 stocks, and the app will not show any historical data (only the current day's) (so the app has data showing only during market hours)



Design Choices

1) Kafka vs pub/sub:

- Some of the stats, like maximum trade amount, rely on a single trade event; thus, missing any event could alter financial calculations by a lot. Thus, we chose Kafka to reliably store and replay any message in case of consumer issues or even producer crashes.
- Currently ignoring that bit of risk when ingesting the message in Python and before committing to Kafka

2) Redis(NoSQL) vs RDBMS:

- The access pattern for the values stored by Flink (timeseries data and single values) plays more to a KV storage solution.

- We also don't need any complex joins of data for the feature set above. These stats are calculated per stock, and that's the only relation.
- There are very frequent writes and relatively similar reads, making an in-memory solution ideal
- We make the process atomic by using Flink checkpointing (with Kafka) and Redis, so no event goes unprocessed in case of a crash from either Flink or Redis.

Initial list of stats to calculate

1. The maximum trade price for the day [single value]
2. The lowest trade price for the day [single value]
3. The price of the last trade for the stock [single value]
4. volume weighted average price throughout the day [graph]
5. simple moving average [graph]

Testing strategy

- Create unit tests for all the stat calculation functions
- Write some integration tests with a bunch of synthetic event data and expected values

Timeline

Progress	Date	Status	Pr's
- Set up the Alpaca API and consume Trade Events - Connect Kafka to the Alpaca API to store Events - Connect Flink to Kafka	Oct 24, 2…	In Prog…	
- Calculate all stats mentioned above - Add Redis and store Stats within it - Add Unit tests for all Stat calculations	Nov 7, 2025	Not St…	
- Integrate the Dashboard UI using Streamlit	Nov 25, 2…	Not St…	
- Make presentation slides and record	Dec 1, 2025	Not St…	

Milestone Goals

I will complete the following goals by 11/10/2025

- Calculate all stats mentioned above
- Add Redis and store Stats within it
- Set up the Alpaca API and consume Trade Events
- Connect Kafka to the Alpaca API to store Events
- Connect Flink to Kafka

Final Goals

I will complete the following goals by 12/5/2025

- Add Unit tests for all Stat calculations
- The Kafka component should hold all events coming through the Alpaca API
- Flink should consume the events in a Kafka topic and calculate all the stats listed above
- Flink will write the values of those stats in Redis
- TickerFlow webapp will read from Redis to populate the UI with those stat values
- Make presentation slides and record